

User Guide

JALoP over BEEP (v1.x)

Draft

19 April 2024

This Page is Intentionally Left Blank

Contents

1	JALoPv1.x.....	7
2	Download/Clone Source Code	7
2.1	GitHub	7
2.2	JALoP CTC-Internal GitLab Repositories.....	7
3	Build and Install C Publisher (jald) & C Subscriber (jal_subscribe)	8
3.1	System Provisioning	8
3.1.1	RHEL 7 / CentOS 7	8
3.1.2	RHEL 8 / CentOS 8	8
3.1.3	RHEL 9	9
3.2	Build and Install C Publisher & C Subscriber	10
3.3	Configure C Publisher.....	11
3.4	Configure JAL-Local-Store	12
3.5	Configure C Subscriber	14
4	Build and Install BeepCore Java	16
4.1	System Provisioning	16
4.2	Clone “beepcore-java” repository from GitHub	16
4.3	Install BeepCore Java from pre-built jar files.....	17
5	Build and Install Java Subscriber and Publisher (jnl_test-*.jar)	17
5.1	Clone “jjnl” repository from GitHub	17
5.2	Build and Install Java Subscriber (jnl_test)	18
5.3	Configure jnl_test-*.jar as a Subscriber	18
5.4	Configure jnl_test-*.jar as a Publisher	19
6	Run Publisher and Subscriber to Transfer JAL Records	20
6.1	Disable SELinux or set to Permissive (For Testing Only)	20
6.2	Stop Firewall (For Testing Only).....	20
6.3	Start JAL-LOCAL-STORE	21
6.4	Insert Records into JAL-Local-Store	21
6.4.1	Insert Log Records.....	21
6.4.2	Insert Audit Records.....	21
6.4.3	Insert Journal Records.....	21
6.5	Check for Inserted Records in JAL-Local-Store.....	21
6.6	Start C Publisher.....	22

6.7	Start Subscriber.....	22
6.8	Purge Records from Local Store.....	23
6.9	Run Java Publisher and Java Subscriber.....	23
6.9.1	Add Records to Java Publisher.....	23
6.9.2	Insert Log Records.....	23
6.9.3	Insert Audit Records.....	23
6.9.4	Insert Journal Records.....	23
6.9.5	Check for Java Publisher Records.....	23
6.9.6	Start Java Publisher.....	24
6.9.7	Start Java Subscriber.....	24
6.10	Run C Publisher (jald) with GDB.....	24
6.11	Run C Publisher (jald) with Valgrind.....	24
7	JALoP Data-Taps.....	24
7.1	jalop-coreutils.....	24
7.1.1	Build & Install.....	25
7.1.2	Run tee and tail.....	25
7.2	jalop-jalauditd.....	25
7.2.1	Build & Install.....	25
7.2.2	Configure jalauditd.....	26
7.2.3	Run jalauditd.....	26
7.3	jalop-rsyslog.....	27
7.3.1	Build & Install.....	27
7.3.2	Configure omjal plugin.....	27
7.3.3	Install omjal plugin.....	28
7.4	jalop-log4cxx.....	29
7.4.1	Build & Install.....	29
7.4.2	Create, Build & Run a Test Executable.....	30
7.5	jalop-jaljournald.....	31
7.5.1	Build & Install.....	31
7.5.2	Configure jaljournald.....	31
7.5.3	Run jaljournald.....	32

This Page is Intentionally Left Blank

1 JALoPv1.x

JALoPv1.x is JALoP over BEEP. It has a C Publisher/Subscriber, a Java Publisher/Subscriber, and several data-taps. The Publisher usually resides on a CDS system to securely and reliably transfer journal, audit, and log records generated by the CDS to one or more remote Subscribers.

2 Download/Clone Source Code

Create a top-level 'jalop' directory to store all JALoP repositories. This will be referred to as <jalop_root> throughout this document.

```
% mkdir jalop
% cd jalop/
```

2.1 GitHub

JALoP Publisher and Subscriber repositories are available on GitHub at this URL – <https://github.com/JALoP>

Clone “JALoP” and “jjnl” repositories using git –

```
% git clone https://github.com/JALoP/JALoP.git
% cd JALoP/
% git checkout -t origin/1.x.x.x

% git clone https://github.com/JALoP/jjnl.git
% cd jjnl/
% git checkout -t origin/1.x.x.x
```

The “jjnl” repository requires Java BEEP Core, which is also available on GitHub.

```
% git clone https://github.com/JALoP/beepcore-java.git
```

2.2 JALoP CTC-Internal GitLab Repositories

All the JALoP source code repositories are available on the CTC-internal gitlab server on the ISIS network. You must VPN into the ISIS network to access those repositories.

If you have been granted access to the internal gitlab repositories as a developer, you can git clone the repositories as below –

```
% git clone git@gitlab.cdsc.ctc.com:jalop/jalop.git
% cd jalop/
% git checkout -t origin/1.x.x.x

% git clone git@gitlab.cdsc.ctc.com:jalop/jjnl.git
% cd jjnl/
% git checkout -t origin/1.x.x.x

% git clone git@gitlab.cdsc.ctc.com:jalop/beepcore-java.git
```

3 Build and Install C Publisher (jald) & C Subscriber (jal_subscribe)

The C Publisher (jald) is the process that negotiates with the Subscriber(s) and sends JAL records to them. This is in “jalop” (or “JALoP” if cloned from GitHub) repository. Clone the repository as mentioned above, if not done yet.

3.1 System Provisioning

3.1.1 RHEL 7 / CentOS 7

The following instructions will allow a CentOS 7 minimal install to be provisioned to build and run the JALoP C implementation:

- Install EPEL
 - `$ sudo yum install epel-release`
- Install JALoP dependencies available from repos
 - `$ sudo yum install @development libxml2-devel libconfig-devel libuuid-devel openssl-devel libdb-devel xmlsec1-openssl-devel python2-scons lcov libtool-ltdl-devel doxygen libseccomp-devel systemd-devel libcap-devel libcap-ng-devel`
- Install bundled JALoP dependencies from provided RPMs
 - Navigate into your cloned JALoP repo
 - Install axl RPM
 - `$ sudo yum install 3rd-party/axl/RHEL7/*.x86_64.rpm`
 - Install vortex RPM
 - `$ sudo yum install 3rd-party/vortex/RHEL7/*.x86_64.rpm`
- Download and install test-dept unit test library.
 - `$ git clone https://github.com/norrby/test-dept.git`
 - `$ cd test-dept`
 - `$ ./bootstrap`
 - `$ ./configure`
 - `$ sudo make install`
- NOTE: axl/vortex build steps below are only needed if these dependencies were not already installed via the provided RPMs in the steps above.
- Build and install axl
 - `$ git clone https://github.com/ASPLes/libaxl.git`
 - `$ cd libaxl`
 - `$ ./autogen.sh`
 - `$ sudo make install`
- Build and install vortex
 - `$ git clone https://github.com/ASPLes/libvortex-1.1.git`
 - `$ cd libvortex-1.1`
 - `$ ./autogen.sh --disable-{sasl,websocket,xml-rpc,tunnel,pull,http,alive}-support --disable-{py,lua}-vortex --disable-vortex-client`
 - `$ sudo make install`

3.1.2 RHEL 8 / CentOS 8

The following instructions will allow a RHEL 8/CentOS Stream 8 minimal install to be provisioned to build and run the JALoP C implementation:

- RHEL 8: Enable CodeReady Builder

- `$ sudo subscription-manager repos --enable codeready-builder-for-rhel-8-x86_64-rpms`
- CentOS 8: Enable Powertools (unbranded version of RHEL's CodeReady Builder)
 - `$ sudo dnf config-manager --set-enabled powertools`
- Install JALoP dependencies available from repos
 - `$ sudo dnf install @development libxml2-devel libconfig-devel loglibuuid-devel openssl-devel libdb-devel xmlsec1-openssl-devel libtool-ltdl-devel apr-util-devel libcurl-devel doxygen lcov libseccomp-devel systemd-devel libcap-devel libcap-ng-devel`
- Install bundled JALoP dependencies from provided RPMs
 - Navigate into your cloned JALoP repo
 - Install axl RPM
 - `$ sudo yum install 3rd-party/axl/RHEL8/*.x86_64.rpm`
 - Install vortex RPM
 - `$ sudo yum install 3rd-party/vortex/RHEL8/*.x86_64.rpm`
- Build and install test-dept
 - `$ git clone https://github.com/norrby/test-dept.git`
 - `$ cd test-dept`
 - `$ ./bootstrap`
 - `$ ./configure`
 - `$ sudo make install`
- NOTE: axl/vortex build steps below are only needed if these dependencies were not already installed via the provided RPMs in the steps above.
- Build and install axl
 - `$ git clone https://github.com/ASPLes/libaxl.git`
 - `$ cd libaxl`
 - `$ ./autogen.sh --disable-py-axl`
 - `$ sudo make install`
- Build and install vortex
 - `$ git clone https://github.com/ASPLes/libvortex-1.1.git`
 - `$ cd libvortex-1.1`
 - `$ ./autogen.sh --disable-{sasl,websocket,xml-rpc,tunnel,pull,http,alive}-support --disable-{py,lua}-vortex --disable-vortex-client`
 - `$ sudo make install`
- Install python36-scons
 - `$ sudo yum install python36-scons`
 - `$ sudo pip3 install scons`
- Use python36 as python
 - `$ sudo alternatives --config python`
 - Enter the number for the /usr/bin/python36 selection
- Ensure /usr/local/lib is in the dynamic linker path
 - `$ echo /usr/local/lib | sudo tee /etc/ld.so.conf.d/usr-local-lib.conf`
 - `$ sudo ldconfig`

3.1.3 RHEL 9

The following instructions will allow a RHEL 9 minimal install to be provisioned to build and run the JALoP C implementation:

- RHEL 9: Enable CodeReady Builder
 - `$ sudo subscription-manager repos --enable codeready-builder-for-rhel-9-x86_64-rpms`

- Install JALoP dependencies available from repos
 - `$ sudo dnf install @development libxml2-devel libconfig-devel libuuid-devel openssl-devel libdb-devel xmlsec1-openssl-devel libtool-ltdl-devel apr-util-devel libcurl-devel doxygen lcov libseccomp-devel systemd-devel libcap-devel libcap-ng-devel`
- Install bundled JALoP dependencies from provided RPMs
 - Navigate into your cloned JALoP repo
 - Install axl RPM
 - `$ sudo yum install 3rd-party/axl/RHEL9/*.x86_64.rpm`
 - Install vortex RPM
 - `$ sudo yum install 3rd-party/vortex/RHEL9/*.x86_64.rpm`
- Build and install test-dept
 - `$ git clone https://github.com/norrby/test-dept.git`
 - `$ cd test-dept`
 - `$ ./bootstrap`
 - `$ ./configure`
 - `$ sudo make install`
- NOTE: axl/vortex build steps below are only needed if these dependences were not already installed via the provided RPMs in the step above.
- Build and install axl
 - `$ git clone https://github.com/ASPLes/libaxl.git`
 - `$ cd libaxl`
 - `$ ./autogen.sh --disable-py-axl`
 - `$ sudo make install`
- Build and install vortex
 - `$ git clone https://github.com/ASPLes/libvortex-1.1.git`
 - `$ cd libvortex-1.1`
 - `$ ./autogen.sh --disable-{sasl,websocket,xml-rpc,tunnel,pull,http,alive}-support --disable-{py,lua}-vortex --disable-vortex-client`
 - `$ sudo make install`
- Install python3-scons
 - `$ sudo yum install python3-scons`
- Ensure /usr/local/lib is in the dynamic linker path
 - `$ echo /usr/local/lib | sudo tee /etc/ld.so.conf.d/usr-local-lib.conf`
 - `$ sudo ldconfig`

3.2 Build and Install C Publisher & C Subscriber

The “jalop” (or “JALoP” if cloned from GitHub) repository builds the following applications –

- “jald” (the JALoP C Publisher)
- “jal-local-store” (the JAL-Local-Store)
- “jaldb_tail”
- “jal_dump”
- “jal_purge”
- “jalp_test” (a test Producer)
- “jal_subscribe” (the JALoP C Subscriber)

It also builds the following shared libraries –

- “libjal-common.so”

- “libjal-db.so”
- “libjal-network.so”
- “libjal-producer.so”
- “libjal-utils.so”

Follow the instructions below to build and install the above-mentioned components -

- Change to the jalop top directory –
 - `% cd <jalop_root>/jalop/` (or, `cd <jalop_root>/JALoP/`, if cloned from github).
- Checkout the “1.x.x.x” branch of jalop (if not already checked out. Check “git branch” output) -
 - `% git checkout 1.x.x.x`
- Clean up and build jalop -
 - `% scons -c` # to clean up.
 - `% scons`
- Install the publisher -
 - `% sudo ./install_rhel_x86_64.sh`

3.3 Configure C Publisher

Make a copy of `<jalop_root>/jalop/test-input/jald.cfg` and adjust according to your Subscriber’s IP, port, etc. This is to avoid changing the sample configuration file in the git source tree. Here is an example of `jald.cfg` file –

```
# The path to the private key, used for TLS.
private_key = "./test-input/cert_and_key";

# The path to the public cert, used for TLS.
public_cert = "./test-input/cert";

# The directory containing the certificates for the remote peers.
remote_cert_dir = "./test-input/certs";

# The path to the root of the database.
db_root = "/root/testdb";

# The path to a directory containing the JALoP schemas.
schemas_root = "./schemas";

# The port the Publisher will listen on.
port = 8444L;

# The IP address (interface) the Publisher will to listen on, or
0.0.0.0 to listen on all.
host = "127.0.0.1";

# For subscribe, the maximum number of records to send before sending
a 'digest' message
pending_digest_max = 10L;
```

```

# For subscribe, the maximum number of seconds to wait, before
sending a 'digest' message
pending_digest_timeout = 100L;

# How long to wait, in seconds, before polling for records after
finding no records
poll_time = 1L;

# A list of supported digest algorithms. These algorithms should be
ordered by preference
# in a single double-quoted string with a space separating the
algorithms.
# Valid values are "sha256", "sha384", and "sha512"
digest_algorithms = "sha256";

# List of allowed Subscriber peer configurations
peers = ( {
    hosts = ("127.0.0.1");
    subscribe_allow = ("journal", "audit", "log");
} );

```

Check the section “Run the Publisher and Subscriber to Transfer JAL Records” below.

3.4 Configure JAL-Local-Store

The JAL-Local-Store (`jal-local-store`) is a process that receives and stores JAL Data sent from the JAL Producer applications. It has a Berkeley Database (BDB) to store and process the JAL records. The `jal-local-store` process must be started before the Publisher process (`jal`).

Make a copy of `<jalop_root>/jalop/test-input/local_store.cfg` and update accordingly. This is to avoid changing the sample configuration file in the git source tree. Here is an example local store configuration file –

```

private_key_file = "./test-input/rsa_key";
public_cert_file = "./test-input/cert";
system_uuid = "34c90268-57ba-4d4c-a602-bdb30251ec77";
hostname = "test.jalop.com";
db_root = "./testdb";
schemas_root = "./schemas/";

# The jal-local-store process, at startup, will check if it is
# running under systemd along with a systemd socket unit file
configuration.
# If so, there is no need to define any socket parameters below, they
will be ignored.
# Systemd will create the socket file for jal-local-store.
#
# If there is no systemd socket, jal-local-store will attempt to
create it.

```

```

# Enter the file system path where the socket file will be created
# socket_owner and socket_group will default to the user and group
# the jal-local-store process is running as and socket mode will
# default to 0666.
socket = "./jal.sock";
#
# Uncomment to define a socket_owner other than the default (the user
# the process is running as)
# The username used must exist on the system.
#socket_owner = "jalls";
#
# Uncomment to define a socket_group other than the default (the
# group the process user belongs to).
# The groupname used must exist on the system.
#socket_group = "jalproducer";
#
# Uncomment to define socket_mode other than the default (0666).
# This must be a string representing exactly 4 digits.
# Each digit must be in the range of 0-7.
#socket_mode = "0420";
#
# example socket file listing after being created
# sr--w----.  1 jalls jalproducer  0 Jan 23 13:15
# /var/run/jalop/jal.sock

# Process will cd to / (root directory), fork, and will run as a
# daemon.
# When running the process as daemon, and even though the jal-local-
# store
# will resolve relative paths for you, it is always safer to use
# absolute paths for configurations in this file that require file
# system paths.
daemon = true;

sign_sys_meta = false;
manifest_sys_meta = false;

# Digest algorithm to use to generate digests in system metadata
sys_meta_dgst_alg = "sha256";

# Flow control functionality turned off if accept_delay_thread_count
# set to zero.
# Below are the default values if not set.
#accept_delay_thread_count = 10;
#accept_delay_increment = 100;
#accept_delay_max = 10000000;

# File storing PID of jal-local-store when daemonized.
#pid_file = "/var/log/jalop/jls-pid.txt";

# Log directory of jal-local-store when daemonized.

```

```

log_dir = ".";

# seccomp will restrict the jal-local-store process to the defined
system calls.
# When the process is in the setup phase, at startup, it will be
restricted to the
# initial_seccomp_rules and final_seccomp_rules system call sets.
After the setup phase and before the process
# is doing its routine work, it will be further restricted to only
the final_seccomp_rules system call set.
#
enable_seccomp = true;
# this rule will restrict the process from setting flags on a file
restrict_seccomp_F_SETFL = true;
initial_seccomp_rules =
["geteuid", "getgid", "capget", "capset", "chmod", "chown", "arch_prctl", "b
ind", "brk", "chdir", "dup2", "execve", "flock", "getcwd", "getdents", "getde
nts64", "getrlimit", "ioctl", "listen", "lstat", "prctl", "prlimit64", "rena
me", "rt_sigaction", "rt_sigprocmask", "seccomp", "select", "set_tid_addre
ss", "setsid", "statfs", "sysinfo"];
final_seccomp_rules =
["sched_yield", "accept", "access", "brk", "clone", "close", "connect", "exi
t", "exit_group", "fcntl", "fdatasync", "fstat", "futext", "getpid", "getppid
", "getrandom", "getsockopt", "gettid", "getuid", "lseek", "madvise", "mkdir
", "mmap", "mprotect", "munmap", "open", "openat", "poll", "pread64", "pwrite
64", "read", "recvmsg", "rt_sigreturn", "set_robust_list", "socket", "stat"
, "unlink", "write"];

```

3.5 Configure C Subscriber

Make a copy of <jalop_root>/jalop/test-input/jal_subscribe.cfg and update accordingly. This is to avoid changing the sample configuration file in the git source tree. Here is an example C subscriber configuration file –

```

# The path to the private key, used for TLS.
private_key = "<repo_root>/test-
input/TLS_CA_Signed/client/jal_subscriber_v1_client.key.pem;

# The path to the public cert, used for TLS.
public_cert = "<repo_root>/test-
input/TLS_CA_Signed/client/jal_subscriber_v1_client.cert.pem;

# Directory containing the certificates for the remote peers.
remote_cert = "<repo_root>/test-
input/TLS_CA_Signed/client/trust_store_dir";

# The path to the root of the database.
db_root = "./testdb";

# The path to a directory containing the JALoP schemas.

```

```

schemas_root = `./schemas`;

# The port to connect on.
port = 1234L;

# The hostname or IP address of the Publisher to subscribe to.
host = "127.0.0.1";

# For subscribe, the maximum number of records to send before sending
a `digest` message
pending_digest_max = 10L;

# For subscribe, the maximum number of milliseconds to wait, before
sending a `digest` message
# 1000L = 1 second
pending_digest_timeout = 1000L;

# The supported record types.
data_class = [ "audit", "log", "journal" ];

# Mode to request data in. May be "archive" or "live"
mode = "archive";

# The time before jal_subscribe ends (HH:MM:SS). Specify 00:00:00 to
run continuously.
session_timeout = "00:00:00";

# This window size for the beep channels created
# will be set to this number * 1024 (1k)
# Beep recommends a minimal size of 4k
window_size = 4;

# A list of supported digest algorithms. These algorithms should be
ordered by preference
# in a single double-quoted string with a space separating the
algorithms.
# Valid values are "sha256", "sha384", and "sha512"
digest_algorithms = "sha256";

# seccomp will restrict the jal_subscribe process to the defined
system calls.
# When the process is in the setup phase, at startup, it will be
restricted to the
# initial_seccomp_rules, both_seccomp_rules, and final_seccomp_rules
system call sets. After the setup phase and before the process
# is doing its routine work, it will be further restricted to only
the both_seccomp_rules and final_seccomp_rules system call sets.
enable_seccomp = false;
seccomp_debug = false;

```

```
initial_seccomp_rules = [ "bind", "recvmsg", "access", "arch_prctl",  
"brk", "execve", "flock", "fstat", "fstatfs", "lseek", "prctl",  
"rt_sigprocmask", "seccomp", "set_tid_address" ];  
both_seccomp_rules = [ "open", "ioctl", "clone", "close", "fcntl",  
"getpid", "mmap", "mprotect", "munmap", "openat", "prlimit64",  
"read", "rt_sigaction", "set_robust_list", "stat", "write" ];  
final_seccomp_rules = [ "fsync", "mkdir", "gettid", "getppid",  
"getuid", "getrlimit", "sched_yield", "rename", "select", "getdents",  
"getpeername", "connect", "epoll_create", "epoll_ctl", "epoll_wait",  
"exit", "exit_group", "fdatasync", "futex", "getsockname", "madvise",  
"nanosleep", "pread64", "pwrite64", "recvfrom", "rt_sigreturn",  
"sendto", "setsockopt", "shutdown", "socket" ];
```

4 Build and Install BeepCore Java

4.1 System Provisioning

Install the following packages (if not done already) –

- Java 8 or newer
- Ant
- Maven

```
% sudo yum install java-1.8.0-openjdk-devel ant maven
```

4.2 Clone “beepcore-java” repository from GitHub

```
% cd <jalop_root>/  
% git clone https://github.com/JALoP/beepcore-java.git
```

Build and Install BeepCore JavaBuild –

```
% cd <jalop_root>/beepcore-java/  
% ant dist-tgz
```

Install –

```
% cd build/beepcore-0.9.20/lib/  
% ./install_jars.sh
```

This should install 3 JAR files to ~/.m2/repository/ as shown in the example output below –

```
[INFO] Installing /home/marefin/jalop/beepcore-java/beepcore.jar to  
/home/marefin/.m2/repository/org/beepcore-  
java/beepcore/0.9.20/beepcore-0.9.20.jar
```

```
[INFO] Installing /home/marefin/jalop/beepcore-java/beeptls-jsse.jar  
to /home/marefin/.m2/repository/org/beepcore-java/beeptls-  
jsse/0.9.20/beeptls-jsse-0.9.20.jar
```

```
[INFO] Installing /home/marefin/jalop/beepcore-java/concurrent.jar to
/home/marefin/.m2/repository/EDU/oswego/cs/dl/util/concurrent/1.3.4/co
ncurrent-1.3.4.jar
```

Verify that the 3 JAR files mentioned above are successfully installed; if for some reason those are not installed, manually install (copy) them to intended destination as shown above.

4.3 Install BeepCore Java from pre-built jar files

Extract –

```
% cd <jalop_root>/jjnl/3rd-Party/beepcore-java/0.9.20
% tar -xvzf beepcore-0.9.20.tgz
```

Install –

```
% cd ./beepcore-0.9.20/lib/
% sh ./install_jars.sh
```

This should install 3 JAR files to ~/.m2/repository/ as shown in the example output below –

```
[INFO] Installing /home/marefin/jalop/beepcore-java/beepcore.jar to
/home/marefin/.m2/repository/org/beepcore-
java/beepcore/0.9.20/beepcore-0.9.20.jar
```

```
[INFO] Installing /home/marefin/jalop/beepcore-java/beeptls-jsse.jar
to /home/marefin/.m2/repository/org/beepcore-java/beeptls-
jsse/0.9.20/beeptls-jsse-0.9.20.jar
```

```
[INFO] Installing /home/marefin/jalop/beepcore-java/concurrent.jar to
/home/marefin/.m2/repository/EDU/oswego/cs/dl/util/concurrent/1.3.4/co
ncurrent-1.3.4.jar
```

Verify that the 3 JAR files mentioned above are successfully installed; if for some reason those are not installed, manually install (copy) them to intended destination as shown above.

5 Build and Install Java Subscriber and Publisher (jnl_test-*.jar)

5.1 Clone “jjnl” repository from GitHub

This is the “jjnl” repository. Clone the repository as below (if not done already) –

```
% cd <jalop_root>/
% git clone https://github.com/JALoP/jjnl.git
% cd jjnl
% git checkout -t origin/1.x.x.x
```

5.2 Build and Install Java Subscriber (jnl_test)

- Build -
 - `% cd <jalop_root>/jjnl/jnl_parent/`
 - `% mvn clean`
 - `% mvn package`
- Install (optional) -
 - `% cd <jalop_root>/jjnl/jnl_lib/`
 - `% mvn install`

Note that “jnl_test-*.jar” can be configured and run either as a Subscriber or as a Publisher, as described in the subsequent sections.

5.3 Configure jnl_test-*.jar as a Subscriber

```
% cd <jalop_root>/jjnl/jnl_test/
```

Copy `./src/test/resources/sampleSubscriber.json` here and update accordingly. This is to avoid changing the sample config file in git source tree. If not running with TLS, comment out or remove the “ssl” section below.

An example of sampleSubscriber.json given below –

```
{
  "address": "127.0.0.1",
  "port": 8444,
  "subscriber": {
    "sessionTimeout": "00:00:00",
    "dataClass": [ "audit", "log", "journal" ],
    "digestAlgorithms": [ "SHA256", "SHA384", "SHA512" ],
    "pendingDigestMax": 1,
    "pendingDigestTimeout": 120,
    "output": "./output",
    "mode": "archive",
  }
  "ssl": {
    "Key Algorithm": "SunX509",
    "Key Store Passphrase": "changeit",
    "Key Store Data Type": "file",
    "Key Store": "./certs/server.jks",

    "Trust Algorithm": "SunX509",
    "Trust Store Passphrase": "changeit",
    "Trust Store Data Type": "file",
    "Trust Store": "./certs/remotes.jks",
  }
}
```

Check the section “Run the Publisher and Subscriber to Transfer JAL Records” below.

5.4 Configure jnl_test-*.jar as a Publisher

Known Limitation: The jnl_test-*.jar Java Publisher does not support batching of digest-challenge. Therefore, set the batch size (pendingDigestMax) to 1 as shown below.

```
{
  "address": "192.168.90.1",
  "port": 1234,
  "configureTLS": "on",
  "listener": {
    "pendingDigestMax": 1,
    "pendingDigestTimeout": 120,
    "input" : "../input",
    "output" : "../output",
    "peers": [
      { "hosts": ["192.168.90.5"],
        "publishAllow": ["audit", "journal", "log"],
        "subscribeAllow": ["audit", "journal", "log"]
      }
    ]
  }
}
"ssl": {
  "Key Algorithm": "SunX509",
  "Key Store Passphrase": "changeit",
  "Key Store Data Type": "file",
  "Key Store":
"./certs/TLS_CA_Signed/server/jal_publisher_v1_server.jks",

  "Trust Algorithm": "SunX509",
  "Trust Store Passphrase": "changeit",
  "Trust Store Data Type": "file",
  "Trust Store":
"./certs/TLS_CA_Signed/server/jal_publisher_v1_server-remotes.jks",
}
}
```

6 Run Publisher and Subscriber to Transfer JAL Records

Follow the steps below to run the C Publisher and Java Subscriber to transfer JAL records.

6.1 Disable SELinux or set to Permissive (For Testing Only)

=====

Permanent: have "SELINUX=disabled" or "SELINUX=permissive" in
"/etc/selinux/config" file, then restart VM.

Temporary: Enter the command "/usr/sbin/setenforce 0"

6.2 Stop Firewall (For Testing Only)

=====

RHEL/CentOS 6.x

Stop: % sudo service iptables stop

Disable: % sudo chkconfig iptables off

```
RHEL/CentOS 7.x, 8.x, 9.x
    Stop: % sudo service firewalld stop
    Or,   % sudo systemctl stop firewalld
    Disable: % sudo systemctl disable firewalld
```

6.3 Start JAL-LOCAL-STORE

```
=====
# May need to clean up the first time. Make sure no jal-local-store is
running. For examples -
$ cd <jalop_root>/jalop/
$ pkill jal-local-store
$ sudo rm -rf ./jal.sock      # remove old JAL socket, if any.
$ sudo rm -rf /root/testdb   # clean up existing <db_root>/ directory.
$ sudo mkdir /root/testdb    # For the first time, create <db_root>/
```

Make a copy of ./test-input/local_store.cfg here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

```
$ sudo ./release/bin/jal-local-store --debug --no-daemon -c
./local_store.cfg &
```

6.4 Insert Records into JAL-Local-Store

```
$ cd <jalop_root>/jalop/
```

6.4.1 Insert Log Records

```
=====
$ sudo ./release/bin/jalp_test -j ./jal.sock -a ~/jalop/jalop/test-
input/sample2.cfg -p ~/jalop/jalop/test-input/big_payload.txt -n 100 -
t l
```

6.4.2 Insert Audit Records

```
=====
$ sudo ./release/bin/jalp_test -j ./jal.sock -a ~/jalop/jalop/test-
input/sample2.cfg -p ~/jalop/jalop/test-input/big_payload.txt -n 100 -
t a
```

6.4.3 Insert Journal Records

```
=====
$ sudo ./release/bin/jalp_test -j ./jal.sock -a ~/jalop/jalop/test-
input/sample2.cfg -p ~/jalop/jalop/test-input/big_payload.txt -n 100 -
t j
```

6.5 Check for Inserted Records in JAL-Local-Store

```
=====
$ sudo ./release/bin/jaldb_tail -n 1000000 -h /root/testdb/ -t l | wc
-l
```

```
$ sudo ./release/bin/jaldb_tail -n 1000000 -h /root/testdb/ -t a | wc
-l
$ sudo ./release/bin/jaldb_tail -n 1000000 -h /root/testdb/ -t j | wc
-l
```

Or,

```
$ sudo ./release/bin/jal_purge -h /root/testdb -b 2030-11-11T11:11:11
-x -t l | wc -l
$ sudo ./release/bin/jal_purge -h /root/testdb -b 2030-11-11T11:11:11
-x -t a | wc -l
$ sudo ./release/bin/jal_purge -h /root/testdb -b 2030-11-11T11:11:11
-x -t j | wc -l
```

6.6 Start C Publisher

```
=====
$ cd <jalop_root>/jalop/
```

Make a copy of ./test-input/jald.cfg here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

```
$ sudo ./release/bin/jald -s -d -c ./jald.cfg --no-daemon 2>&1 | tee
publisher.log
```

Enter "man jald" for help.

6.7 Start Subscriber

```
=====
Java subscriber (jnl_test)
=====
$ cd <jalop_root>/jjnl/jnl_test/
```

Make a copy of ./jnl_test/target/test-classes/sampleHttpSubscriber.json here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

```
java -jar target/jnl_test-1.1.0.1.jar ./sampleSubscriber.json 2>&1 |
tee subscriber.log
```

```
=====
or C subscriber (jal_subscribe)
=====
$ cd <jalop_root>/jalop
```

Make a copy of ./test-input/jal_subscribe.cfg here and update accordingly. This is to avoid changing the sample configuration file in the git source tree. If running on same machine as publisher, you need to specify a different location for the database. The publisher and subscriber need to use separate databases.

```
sudo ./release/bin/jal_subscribe -s -d -c ./jal_subscribe.cfg 2>&1 |
tee subscriber.log
```

Enter "man jal_subscribe" for help.

6.8 Purge Records from Local Store

```
=====
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b 2030-11-
11T11:11:11 -t l
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b 2030-11-
11T11:11:11 -t a
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b 2030-11-
11T11:11:11 -t j
```

OR,

```
$ now=$(date -u "+%Y-%m-%dT%H:%M:%S.%N")
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b "$now" -t l
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b "$now" -t a
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b "$now" -t j
```

6.9 Run Java Publisher and Java Subscriber

```
=====
```

Follow the steps below to run the Java Publisher and Java Subscriber to transfer JAL records.

6.9.1 Add Records to Java Publisher.

```
$ cd <jjnl_root>/
$ mkdir input
```

6.9.2 Insert Log Records

```
=====
$ python generate_records.py log input/ 100 ~/jalop/jalop/test-
input/sample2.cfg ~/jalop/jalop/test-input/big_payload.txt
```

6.9.3 Insert Audit Records

```
=====
$ python generate_records.py audit input/ 100 ~/jalop/jalop/test-
input/sample2.cfg ~/jalop/jalop/test-input/big_payload.txt
```

6.9.4 Insert Journal Records

```
=====
$ python generate_records.py journal input/ 100 ~/jalop/jalop/test-
input/sample2.cfg ~/jalop/jalop/test-input/big_payload.txt
```

6.9.5 Check for Java Publisher Records

```
=====
```

```
$ ls <jjnl_root>/input/audit
$ ls <jjnl_root>/input/journal
$ ls <jjnl_root>/input/log
```

6.9.6 Start Java Publisher

```
=====
```

```
$ cd <jjnl_root>/jnl_test/
```

Make a copy of `./src/test/resources/sampleListener.json` here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

```
$ java -jar jnl_test-1.1.01.jar sampleListener.json
```

6.9.7 Start Java Subscriber

```
=====
```

```
$ cd <jalop_root>/jjnl/jnl_test/
```

Make a copy of `./jnl_test/target/test-classes/sampleSubscriber.json` here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

```
java -jar target/jnl_test-1.1.0.1.jar ./sampleSubscriber.json 2>&1 |
tee subscriber.log
```

6.10 Run C Publisher (jald) with GDB

```
=====
```

```
$ export LD_LIBRARY_PATH="/home/marefin/jalop/jalop/debug/lib/"
$ gdb -ex=r --args ./debug/bin/jald -d -s -c ./jald.cfg --no-daemon
```

6.11 Run C Publisher (jald) with Valgrind

```
=====
```

```
$ valgrind --tool=memcheck --leak-check=full --verbose --track-
origins=yes --log-file=valgrind_out.txt ./debug/bin/jald -d -s -c
./jald.cfg --no-daemon
```

7 JALoP Data-Taps

The JALoP data-taps capture and send JAL records to the JALoP Local Store (jal-local-store). Each data-tap depends on JALoP libraries, so JALoP v1 or v2 libraries must be build and installed prior to building a data-tap repository. Generally, each of the data-taps requires the JALoP socket and db_root addresses that jal-local-store (JLS) uses. Note that jal-local-store must be already running for any data-tap to send JAL records to it via the socket.

7.1 jalop-coreutils

Available on IntelLink and JALoP development Git Server.

This has the JALoP version of GNU tail and GNU tee commands.

7.1.1 Build & Install

```
% cd <jalop_root>/jalop-coreutils/  
% autoreconf -fiv #for RHEL7  
% ./bootstrap --skip-po #for RHEL8/RHEL9  
% ./configure --disable-gcc-warnings  
% make -j  
% make check  
% [[ -e /tmp/install ]] || mkdir /tmp/install  
% make install DESTDIR=/tmp/install  
% cp /tmp/install/usr/local/bin/{tee,tail} /usr/local/bin
```

7.1.2 Run tee and tail

Note that the `jal-local-store (JLS)` process must be running already to receive and insert records.

```
# Based on <db_root> location, user may need 'sudo' in the  
following instructions.  
  
% echo "This is a test message to tee into JLS" | tee -j --  
path=<path to JALoP socket> --schema-root=<path to JALoP schema  
directory>  
% echo -e "Test message1\nTest message2" > file_to_tail  
$ tail -j --path=<path to JALoP socket> --schema-root=<path to  
JALoP schema directory> ./file_to_tail
```

Use `jald_db_tail` to check for newly inserted records -

```
% jald_db_tail -t 1 -h <db_root>
```

Use `jal_dump` to verify that the log records came from auditd -

```
% jal_dump -u <UUID found using jald_db_tail> -h <db_root> -t 1 -d  
z
```

7.2 jalop-jalauditd

Available on the NCDSMO IntelLink site and JALoP development Git Server. This is also available at [GitHub.com/JALoP/JALoP-Auditd-Plugin](https://github.com/JALoP/JALoP-Auditd-Plugin).

7.2.1 Build & Install

Note that JALoP v1 or v2 must be built and installed before attempting to build this data tap.

```
% cd <jalop_root>/jalop-jalauditd (or JALoP-Auditd-Plugin)  
% make clean  
% make  
% sudo make install
```

This shall install the followings –

- The binary `/sbin/jalauditd` (or `/usr/sbin/jalauditd`)
- The configuration files for RHEL7 and RHEL8/RHEL9 go into different directories. Make install will install the configuration files in the following locations and will need to be copied to appropriate directory:

- /etc/jalauditd/jalauditd.conf
 - /etc/audit/plugins.d/audisp-jalauditd.conf
- For RHEL7 the configuration files should be:
 - /etc/audisp/jalauditd.conf
 - /etc/audisp/plugins.d/audisp-jalauditd.conf
- For RHEL8/RHEL9 the configuration files should be:
 - /etc/jalauditd/jalauditd.conf
 - /etc/audit/plugins.d/audisp-jalauditd.conf
- The jalauditd.conf file is initially empty; you will need to edit this file, see below.

7.2.2 Configure jalauditd

The configuration file jalauditd.conf is initially empty. There can be 4 settings in this file as shown below.

```
socket = "/path/to/jalop/socket";
schemas = "/path/to/schemas/root";
keypath = "/path/to/key";
certpath = "/path/to/cert";
```

If the socket and schemas locations are not specified above, default locations specified by the JAL Producer Library (JPL) will be used. Below are the default socket and schemas locations –

```
socket = "/var/run/jalop/jalop.sock"
schemas = "/usr/share/jalop/schemas"
```

If keypath or certpath are not specified, no key or cert will be used for signing.

IMPORTANT: These settings must be consistent with the jal-local-store configuration file.

7.2.3 Run jalauditd

Note that the jal-local-store (JLS) process must run to receive and insert records sent by jalauditd via the socket.

Restarting (as root or sudo) auditd automatically runs the jalauditd child process.

```
% service auditd restart (or, systemctl restart auditd)
```

Use jaldb_tail to check for newly inserted records -

```
% sudo jaldb_tail -f -t 1 -h <db_root>
```

Use jal_dump to verify that the log records came from auditd -

```
% sudo jal_dump -u <UUID found using jaldb_tail> -h <db_root> -t 1
-d z
```

See <jalop_root>/jalop-jalauditd/README file for more details.

7.3 jalop-rsyslog

Available on IntelLink and JALoP development Git Server.

7.3.1 Build & Install

Note that JALoP must be built and installed before attempting to build the rsyslog plugin.

```
$ sudo yum install git automake libtool libestr-devel
```

If you have access to a libfastjson-devel package, you can install that and skip to **Error! Reference source not found.**:

```
$ sudo yum install libfastjson-devel
```

7.3.1.1 Libfastjson headers

If you cannot install libfastjson-devel, the libfastjson headers must be made available to the rsyslog plugin in another way.

```
$ git clone https://github.com/rsyslog/libfastjson.git
$ cd libfastjson
$ sudo mkdir /usr/include/libfastjson
$ sudo cp ./*.h /usr/include/libfastjson
$ sudo ln -s /usr/lib64/libfastjson.so.4
/usr/lib64/libfastjson.so
```

On RHEL 7 / CentOS 7:

```
$ export JSON_C_CFLAGS="-g"
$ export JSON_C_LIBS="/usr/lib64/libfastjson.so"
```

On RHEL 8 / CentOS Stream 8:

```
$ export LIBFASTJSON_CFLAGS="-g"
$ export LIBFASTJSON_LIBS="/usr/lib64/libfastjson.so"
```

7.3.1.2 Build omjal plugin

Build the omjal plugin:

```
$ cd jalop-rsyslogd
$ export CPPFLAGS="-I/usr/include/libfastjson"
$ sh autogen.sh
$ make
```

7.3.2 Configure omjal plugin

Add the following configuration section to `/etc/rsyslog.conf`:

```
$ModLoad omjal # calls JAL for rsyslog pass through to JALoP
$ActionOmjalSocket /var/run/jalop/jalop.sock
$ActionOmjalSchemas /share/jalop/schemas
$ActionOmjalKey <path_to_key> # Optional
```

```
$ActionOmjalCert <path_to_key> # Optional
*.* :omjal:
```

\$ModLoad omjal

States that the omjal plugin shall be loaded

\$ActionOmjalSocket /var/run/jalop/jalop.sock

Specifies the socket to be used to forward the messages to the JALoP local store. If this option is omitted from the file, it will use the default value specified in the JALoP library.

\$ActionOmjalSchemas /share/jalop/schemas

Specifies the directory where the schema information is located. If this option is omitted from the file, it will use the default value specified in the JALoP library.

\$ActionOmjalKey <path_to_key>

Specifies the path to the key to be used for signing. If this option is omitted from the file, then no key will be used.

\$ActionOmjalCert <path_to_key>

Specifies the path to the certificate to be used for signing. If this option is omitted from the file, then no certificate will be used.

***.* :omjal;;logJAL**

<msg_filter_expression>:<output_module>:<parameters>;<template_name>

. specifies that all messages will be sent to this module. If, for example, you wanted to handle mail messages, mail.* would handle that.

See <https://www.rsyslog.com/doc/v8-stable/configuration/filters.html> for more information.

7.3.3 Install omjal plugin

Install the omjal plugin:

```
$ cd jalop-rsyslogd
$ make install
```

The rsyslog package installs libraries to /usr/lib64/rsyslog. If your environment uses a different location, either manually copy the library from .libs/omjal.so or change the libdir flag in autogen.sh

After installation, and after starting the **jal-local-store**, restart the rsyslog service:

```
$ sudo systemctl restart rsyslog
```

To ensure that rsyslogd is connecting to jal-local-store, you can run jal-local-store with the debug (-d) flag. When jal-local-store is ready, the message `Thread_count: 1` will appear. After rsyslogd is connected, the message `Thread_count: 2` will appear.

To create messages in rsyslogd, the logger command can be used. For example:

```
logger "This is your message content."
```

Use `jaldb_tail` to check for newly inserted records -

```
% sudo jaldb_tail -f -t 1 -h <db_root>
```

Use `jal_dump` to verify that the log records came from `auditd` -

```
% sudo jal_dump -u <UUID found using jaldb_tail> -h <db_root> -t  
1 -d z
```

7.4 jalop-log4cxx

Available on IntelLink and JALoP development Git Server.

7.4.1 Build & Install

Note that JALoP must be built and installed before attempting to build the `jalop-log4cxx` library.

```
$ sudo yum install apr-util-devel
```

On RHEL 7 / CentOS 7:

```
$ sudo yum install cmake3  
$ sudo yum install centos-release-scl  
$ sudo yum install devtoolset-8-gcc-c++ --  
enablerepo='centos-scl-rh'  
$ sudo scl enable devtoolset-8 'bash'
```

On RHEL 8 / CentOS 8/ RHEL 9:

```
$ sudo yum install cmake
```

Build the library:

On RHEL 7 / CentOS 7:

```
$ cd jalop-log4cxx  
$ cmake3 -D CMAKE_CXX_COMPILER=/opt/rh/devtoolset-  
8/root/usr/bin/g++ -D CMAKE_INSTALL_PREFIX=/usr CMakeLists.txt  
$ make
```

On RHEL 8 / CentOS 8 / RHEL 9:

```
$ cd jalop-log4cxx  
$ cmake -D CMAKE_INSTALL_PREFIX=/usr CMakeLists.txt  
$ make
```

Install the library:

```
$ sudo make install
```

To uninstall the library:

```
$ sudo make uninstall
```

7.4.2 Create, Build & Run a Test Executable

An example test executable is created during the CMake build at `/path/to/jalop-log4cxx/src/examples/cpp/jalp-appender`. To test using this executable, skip ahead to the instructions on starting the `jal_local_store` and running the executable.

To create a new test executable, do the following:

Create a separate folder for the test executable. This can be separate from the `jalop-log4cxx` folder:

```
$ mkdir ~/log4cxx-test
$ cd ~/log4cxx-test
```

Copy required files from `jalop-log4cxx`:

```
$ cp /path/to/jalop-log4cxx/src/main/cpp/jalpappender.cpp .
```

Create a `main.cpp` for the test executable. Use the following as an example:

```
#include <log4cxx/logger.h>
#include <log4cxx/propertyconfigurator.h>
#include <log4cxx/jalpappender.h>

using namespace log4cxx;

LoggerPtr logger(Logger::getLogger("Test"));

int main(int argc, char **argv)
{
    PropertyConfigurator::configure(argv[1]);
    LOG4CXX_INFO(logger, "qwerty")
    return 0;
}
```

Build the test executable:

```
$ g++ main.cpp jalpappender.cpp -llog4cxx -ljaval-producer -o
log4cxx-test
```

Create a configuration file for the test executable. Use the following as a starting point:

```
log4j.rootLogger=DEBUG,A1

log4j.appender.A1=org.apache.log4j.JalpAppender

log4j.appender.A1.layout=org.apache.log4j.PatternLayout
log4j.appender.A1.layout.ConversionPattern=%-4r [%t] %-5p %c %x -
%m%n
```

The following are optional parameters than can be added to the `log4cxx` config file:

After starting the *jal-local-store*, run the test executable:

```
$ ./log4cxx-test path/to/test.cfg
```

This will send one record to jal-local-store before closing the connection. Verify the record with the following:

```
$ jaldb_tail -h /path/to/testdb -t 1 -d a
```

```
log4j.appender.A1.PATH - The path to jalop.sock
log4j.appender.A1.HOSTNAME - The hostname of the machine that the
JALoP local store is on
log4j.appender.A1.APPNAME - The name of the process that is logging
data
log4j.appender.A1.SCHEMA_ROOT - The path to the JALoP schemas
log4j.appender.A1.KEY_PATH - The path to the RSA key used for signing
log4j.appender.A1.CERT_PATH - The path to the Certificate used for
signing
```

This will print the app metadata for the records. The record inserted by the test executable does not create a payload from the string in LOG4CXX_INFO. It ends up in the message tag in the app metadata.

7.5 jalop-jaljournal

Available on NCDSMO IntelLink site and JALoP development GitLab Server.

7.5.1 Build & Install

Dependencies: JALoP v1 or v2 shared libraries must be built and installed.

- /usr/lib64/<jalop shared object files>
- /usr/include/<jalop headers>
- systemd-devel rpm

```
% cd <jalop_root>/jalop-jaljournal
% make clean
% make
% sudo make install
```

This shall install the followings –

- The binary /bin/jaljournal
- The jaljournal configuration file /etc/jalop/jaljournal.cfg. This file initially has some default values; you will need to edit this file, see below.

7.5.2 Configure jaljournal

The jalauditd configuration file /etc/jalauditd/jalauditd.conf initially has some default values; The settings in this file as shown below.

```
# verbose logging
```

```

debug = false;

# Start collecting logs from head, boot, or tail. Default: tail
# head: will get all journal logs available from multiple boots
#       and then continuously poll to get all logs as they arrive
# boot: will get all logs from current boot and then continuously
#       poll to get all logs as they arrive.
# tail: will continuously poll to get all logs as they arrive.
init_point = "tail";

# collect all journal fields. Default: MESSAGE only
metadata = false;

# socket path that jal-local-store will be listening.
socket = "/path/to/jalop/socket";

# jalop schema path
schemas = "/path/to/schemas/root";

# hostname to record in log
hostname = "localhost";

# appname to record in log
appname = "jalop-journald";

# poll delay between checking for new journal logs
# in milliseconds 1000 = 1 second
delay = 1000;

```

If the `socket` and `schema_path` locations are not specified above, default locations specified by the JAL Producer Library (JPL) will be used. Below are the default `socket` and `schema_path` locations –

```

socket = "/var/run/jalop/jalop.sock"
schema_path = "/etc/jalop/schemas"

```

IMPORTANT: These settings must be consistent with the `jal-local-store` configuration file.

7.5.3 Run `jaljournald`

Note that the `jal-local-store` (JLS) process must run to receive and insert records sent by `jaljournald` via the `socket`. Also, make sure the `jal-local-store` `socket` path matches the `jaljournald` `socket` path and you have write permissions on that `socket` path.

Example command-line

```
% jaljournald -c <path-to-config> --debug --metadata
```

Use `jalldb_tail` to check for newly inserted records -

```
% sudo jalldb_tail -f -t 1 -h <db_root>
```

Use `jal_dump` to verify that the log records came from `journald` -
 `% sudo jal_dump -u <UUID found using jaldb_tail> -h <db_root> -t 1`
`-d z`